BIXOLON®

API 레퍼런스 가이드

BXL SDK for UPOS Compliant Android

Ver. 2.07

http://www.bixolon.co.kr

목차

1. 매뉴얼 안내 5

2. 지원 OS 및 인터페이스 6
2-1 운영체제 6
2-2 지원 장치 및 인터페이스 6

3. 개발 환경 7
3-1 환경 설정 7
3-2 Android 장치 연결 7
3-2-1 Bluetooth 7
3-2-2 Network 8
3-2-3 Wi-Fi Direct 9
3-2-4 USB 10
3-2-5 Android 장치 개발자 옵션 설정 12

4. 패키지 콘텐츠 13
4-1 매뉴얼 13
4-2 라이브러리 13
4-3 샘플 소스 코드 13

5 상수값(Defines) 14
5-1 JposException 14
5-2 Event 15
5-2-1 StatusUpdate Event 15
5-2-2 Error Event 15
5-2-3 OutputComplete Event 15
5-2-4 Data Event 15
5-2-5 DirectIO Event 15
5-3 EscapeSequence 19
5-4 Transaction Print 21
5-5 Alignment 21
5-6 바코드 타입 22
5-7 바코드 텍스트 위치 23
5-8 장치 모델명 23
5-9 페이지 모드 인쇄 방향 24
5-10 MSR 암호화 24
5-11 SCR 모드 24
5-12 Character Set 25

6. 각 클래스별 기능 정리 26
6-1 BXLConfigLoader Class 26
6-1-1 openFile() 26
6-1-2 newFile() 26
6-1-3 getEntries() 27
6-1-4 addEntry() 28
6-1-5 removeEntry() 29
6-1-6 saveFile() 30

6-2 POSPrinter Class....31
6-2-1 open()....31
6-2-2 claim()....32
6-2-3 setDeviceEnabled()....33
6-2-4 release()....33
6-2-5 close()....34
6-2-6 checkHealth()....34
6-2-7 setAsyncMode()....35
6-2-8 setCharacterSet()....36
6-2-9 setCharacterEncoding()....37
6-2-10 cutPaper()....38
6-2-11 printBarCode()....39
6-2-12 printBitmap()....40
6-2-13 printBitmap()....41
6-2-14 printNormal()....42
6-2-15 setPageModePrintArea()....43
6-2-16 setPageModePrintDirection()....44
6-2-17 pageModePrint()....45
6-2-18 setPageModeHorizontalPosition()....46
6-2-19 setPageModeVerticalPosition()....47
6-2-20 transactionPrint()....48
6-2-21 displayString()....49
6-2-22 cleanScreen()....50
6-2-23 storeImageFile()....51
6-2-24 displayImage()....52
6-2-25 clearImage()....53
6-2-26 directIO()....54
6-2-27 markFeed()....55
6-2-28 setBitmap()....56
6-2-29 printSvg()....57
6-2-30 updateFrimware()....58
6-3 MSR Class....59
6-3-1 open()....59
6-3-2 claim()....60
6-3-3 setDeviceEnabled()....61
6-3-4 release()....62
6-3-5 close()....62
6-3-6 setAutoDisable()....63
6-3-7 setDataEventEnabled()....64
6-3-8 setDataEncryptionAlgorithm()....65
6-3-9 getTrack1Data()....66
6-3-10 getTrack2Data()....67
6-3-11 getTrack3Data()....68
6-4 SmartCardRW Class....69
6-4-1 open()....69
6-4-2 claim()....70
6-4-3 setDeviceEnabled()....71
6-4-4 release()....71
6-4-5 close()....72
6-4-6 setSCSlot()....73
6-4-7 setIsoEmvMode()....74
6-4-8 beginInsertion()....75

6-4-9 endInsertion() 76
6-4-10 beginRemoval() 77
6-4-11 endRemoval() 78
6-4-12 readData() 79
6-5 CashDrawer Class 80
6-5-1 open() 80
6-5-2 claim() 81
6-5-3 setDeviceEnabled() 82
6-5-4 release() 82
6-5-5 close() 83
6-5-6 openDrawer() 83
6-5-7 getDrawerOpened() 84
6-6 LocalSmartCardRW Class 85
6-6-1 open() 85
6-6-2 claim() 86
6-6-3 setDeviceEnabled() 87
6-6-4 release() 87
6-6-5 close() 88
6-6-6 getCardInfo() 89
6-6-7 getTrack2() 90
6-6-8 getCardNumber() 91
6-6-9 getCardDueDate() 92

7. 사용 예제 93
7-1 텍스트 인쇄 93
7-2 이미지 인쇄 94
7-3 Page mode 인쇄 95

1. 매뉴얼 안내

- 이 SDK 매뉴얼에는 Android용 응용 프로그램 개발에 필요한 Library 에 대한 내용이 기술 되어 있습니다.

- SDK의 사용방법, 사양, 제약에 대해 설명합니다.

2. 지원 OS 및 인터페이스

2-1 운영체제

• 이 소프트웨어는 아래 운영체제를 지원합니다.
• Android 4.0 (Ice Cream Sandwich) 이상

2-2 지원 장치 및 인터페이스

Models	Interface
SPP-R200II	Bluetooth / WLAN / USB
SPP-R200III	Bluetooth / WLAN / USB
SPP-R210	Bluetooth / WLAN / USB
SPP-R215	Bluetooth / USB
SPP-R220	Bluetooth / BLE / WLAN / USB
SPP-R300	Bluetooth / WLAN / USB
SPP-R310	Bluetooth / WLAN / USB
SPP-R400	Bluetooth / WLAN / USB
SPP-R410	Bluetooth / BLE / WLAN / USB
SPP-R418	Bluetooth / BLE / WLAN / USB
SRP-350plusIII	Bluetooth / WLAN / Ethernet / USB
SRP-352plusIII	Bluetooth / WLAN / Ethernet / USB
SRP-350III	Ethernet / USB
SRP-352III	Ethernet / USB
SRP-F310II	Bluetooth / WLAN / Ethernet / USB
SRP-F312II	Bluetooth / WLAN / Ethernet / USB
SRP-F313II	Bluetooth / WLAN / Ethernet / USB
SRP-380	Bluetooth / WLAN / Ethernet / USB
SRP-382	Bluetooth / WLAN / Ethernet / USB
SRP-383	Bluetooth / WLAN / Ethernet / USB
SRP-330II	Ethernet / USB
SRP-332II	Ethernet / USB
SRP-S300	Bluetooth / WLAN / Ethernet / USB
SRP-340II	Ethernet / USB
SRP-342II	Ethernet / USB
STP-103III	USB
SRP-275III	Ethernet / USB
SRP-Q200	Bluetooth / WLAN / Ethernet / USB
SRP-Q300	Bluetooth / WLAN / Ethernet / USB
SRP-Q302	Bluetooth / WLAN / Ethernet / USB
SRP-QE300	Ethernet / USB
SRP-QE302	Ethernet / USB
SRP-E300	Ethernet / USB
SRP-E302	Ethernet / USB
BK3-3	USB

※ BLE: Bluetooth Low Energy

3. 개발 환경

3-1 환경 설정

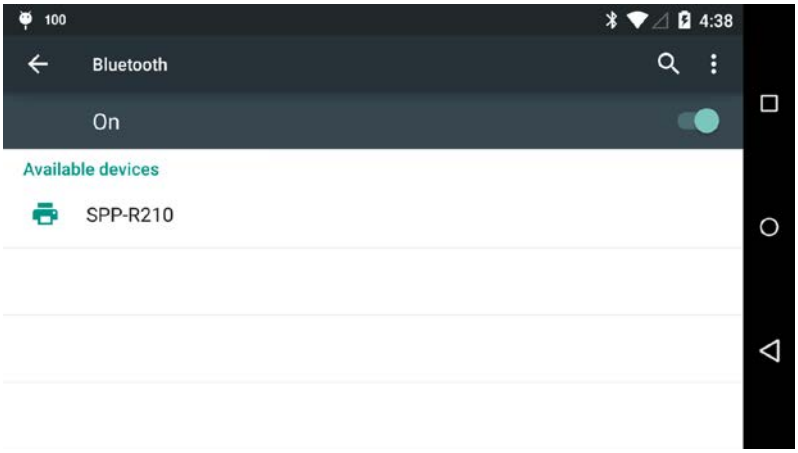
- Java Development Kit (JDK) 7
- Eclipse
- Android SDK Tools
- 참고: http://developer.android.com/sdk/index.html

3-2 Android 장치 연결

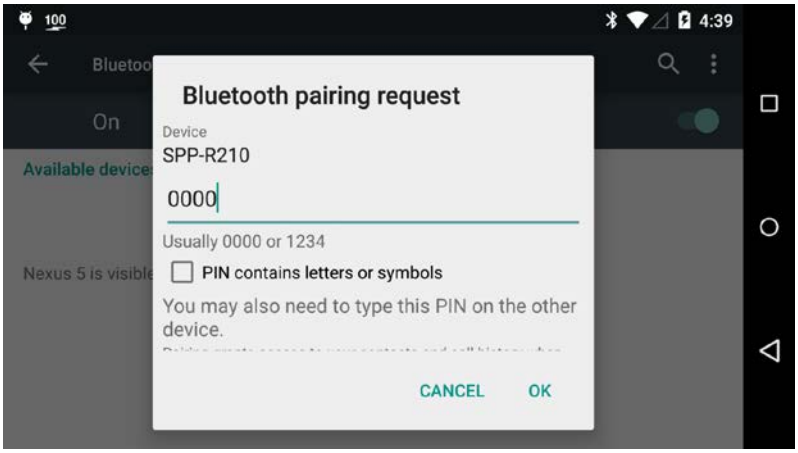
- 아래 스크린샷은 Nexus 5에서 캡쳐한것입니다.
  스크린샷과 항목이름은 Android 운영체제 또는 장치에 따라 다를 수 있습니다.

3-2-1 Bluetooth

1. Settings를 선택합니다.
2. 반드시 Bluetooth가 켜져 있고, 프린터의 전원이 켜져 있어야 합니다.
3. 설정을 위해 Bluetooth를 선택합니다.



4. Scan을 선택합니다. 연결하기 위한 프린터를 검색하고 페어링합니다.
5. PIN code를 입력합니다. ㈜빅솔론의 초기 PIN code는 “0000” 입니다.



3-2-2 Network

1. 프린터를 네트워크 AP(Access Point)에 연결하고 IP 주소를 할당하거나 DHCP로 설정합니다. ㈜빅솔론의 프린터는 초기에 Ad-hoc으로 설정되어 있기 때문에, 당사 Net Configuration Tool로 최초 한번은 설정을 해야 됩니다. Net Configuration Tool은 ㈜빅솔론의 웹사이트에서 다운로드 가능합니다.
   (설정과 관련된 자세한 사항은 Net Configuration Tool 매뉴얼 참고)

2. Settings를 선택합니다.

3. Wi-Fi가 켜져 있어야 합니다.

4. ㈜빅솔론의 프린터가 연결되어있는 네트워크와 동일한 네트워크에 연결합니다.

5. 안드로이드 장치를 프린터의 TCP/IP 포트에 연결하기 위해 추가 설정은 필요하지 않습니다.

3-2-3 Wi-Fi Direct

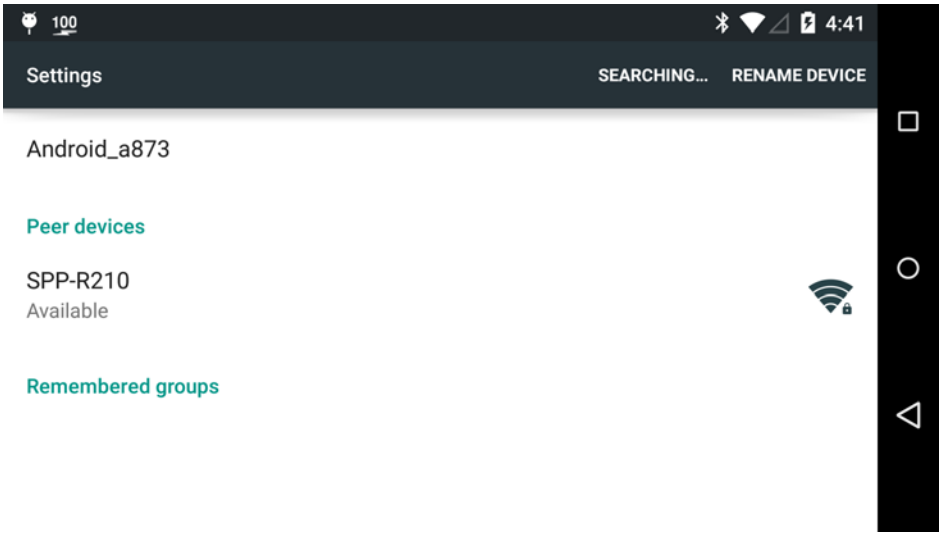
1. 안드로이드 장치는 OS버전 4.0이상부터 Wi-Fi Direct로 주변장치 연결이 가능합니다.

2. 안드로이드 장치에 ㈜빅솔론의 특정 드라이버나 프린터 소프트웨어가 설치 되어 있을 필요는 없습니다.

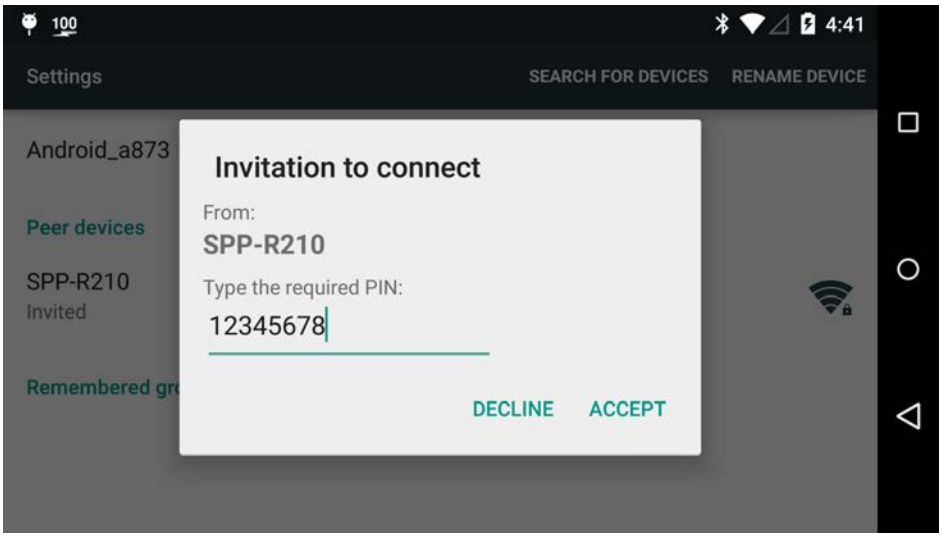
3. Settings를 선택합니다.

4. Wi-Fi가 켜져 있어야 합니다.

5. Wi-Fi Direct를 선택합니다.



6. 검색된 프린터를 선택해서 연결합니다.
   Wi-Fi Direct 초기 PIN 번호는 “12345678” 입니다.



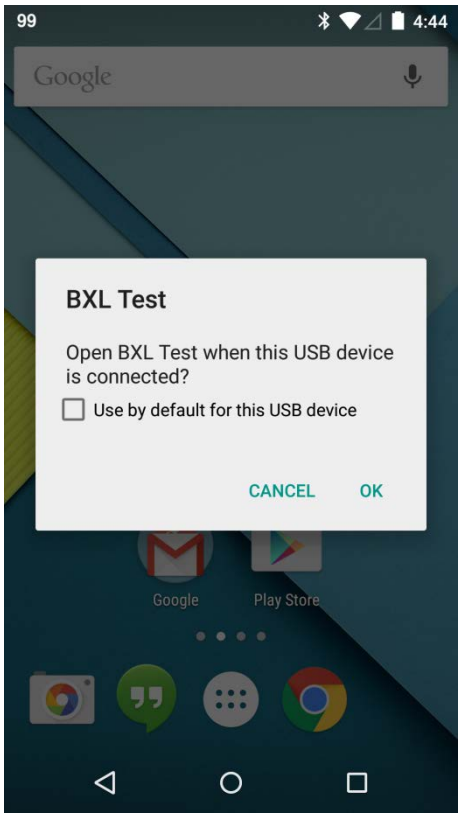
3-2-4 USB

1. 안드로이드 장치는 OS 버전 3.1 이상부터 USB 주변장치 연결이 가능합니다.

2. 안드로이드 장치에 ㈜빅솔론의 특정 드라이버나 프린터 소프트웨어가 설치 되어있을 필요는 없습니다.

3. 필요한 USB 케이블은 안드로이드 장치에 따라 달라질 수 있습니다.
   사용하려는 안드로이드 장치가 USB 연결을 지원하는지 확인 후 알맞은 케이블을 사용하시기 바랍니다.

4. ㈜빅솔론의 프린터를 처음 연결하는 경우, 안드로이드 장치에 따라서 아래와 같은 메시지가 나타날 수 있습니다.



5. USB 주변장치를 연결하기 위해서는 아래 코드를 AndroidManifest.xml과 Sample로 제공되는 BXLTest내 res/xml/device_filter.xml에 입력해야 합니다.

[AndroidManifest.xml]

```
…
<uses-feature android:name="android.hardware.usb.host" />
…

<intent-filter>
        <action    android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
/>
</intent-filter>

<meta-data
        android:name="android.hardware.usb.action.USB_DEVICE_ATTACHED"
        android:resource="@xml/device_filter" />
```

[device_filter.xml]

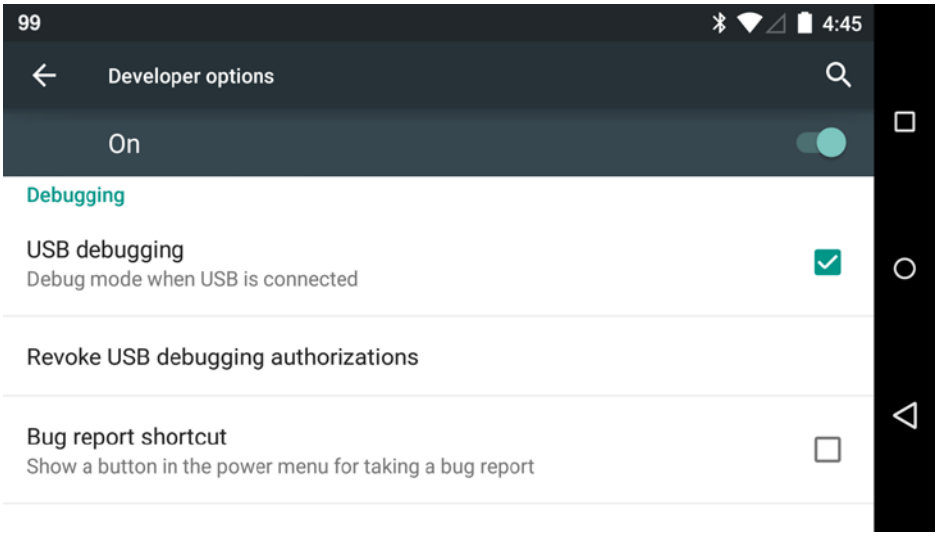
```
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <usb-device
        class="7"
        protocol="2"
        subclass="1" />

</resources>
```

3-2-5 Android 장치 개발자 옵션 설정

1. Settings를 선택합니다.

2. Developer options를 선택합니다.

3. USB debugging을 활성화합니다.



4. 패키지 콘텐츠

4-1 매뉴얼

매뉴얼 위치/이름	설명
docs/Manual_BXL SDK for Android_UPOS compliant API Reference Guide_english_Ver_x.xx	영어 매뉴얼
docs/Manual_BXL SDK for Android_UPOS compliant API Reference Guide_japan_Ver_x.xx	일본어 매뉴얼
docs/Manual_BXL SDK for Android_UPOS compliant API Reference Guide_한국어_Ver_x.xx	한국어 매뉴얼
docs/Manual_BXL SDK for Android_UPOS compliant API Reference Guide_chinese_Ver_x.xx	중국어 매뉴얼

4-2 라이브러리

라이브러리 위치/이름	설명
libs/bixolon_printer_Vxxx.jar	JavaPOS 서비스 컴포넌트 계층의 구현. 프린터 설정 라이브러리
libs/jpos11x-controls.jar	JavaPOS 장치 컨트롤과 서비스 인터페이스
libs/xerces.jar	apache.org XML 서비스구현 (JavaPOS device control에서 필요)
libs/icu4j-58_1.jar	아랍어/페르시아어 문자를 인쇄하기 위해 사용되는 라이브러리
libs/Image/OpenCV-2.4.11.jar	Image 인쇄를 위한 라이브러리
libs/Image/CPU type/libopencv_java.so	Image 인쇄를 위한 native 라이브러리

4-3 샘플 소스 코드

샘플 위치/이름	설명
samples/BixolonSample	프린터/MSR/SRC/CashDrawer 사용 샘플 애플리케이션

★ SDK 활용 이해를 높이기 위한 샘플 레퍼런스 클래스 제공
- BixolonPrinter.java(src\main\java\com\bixolon\sample\PrinterControl)

5 상수값(Defines)

5-1 JposException

• Method에서 특정 기능 수행 중 에러 발생시 JposException 예외를 발생 시킵니다.
예외 발생 시 “printStackTrace” 함수를 통해 에러 내용을 확인할 수 있습니다.

[Example]

```
import jpos.JposConst;

try
{
	……
}
catch(JposException e)
{
	// Error
	e.printStackTrace();
}
```

5-2 Event

• 각 Event는 JposConst, POSPrinterConst 클래스에 정의되어 있습니다.

5-2-1 StatusUpdate Event

프린터 상태가 변경될 때 마다 StatusUpdate 이벤트가 발생합니다.

상수명	값	설명
JPOS_SUE_POWER_ONLINE	2001	Printer Power on
JPOS_SUE_POWER_OFF_OFFLINE	2004	Printer Power off
PTR_SUE_COVER_OPEN	11	Cover Open
PTR_SUE_COVER_OK	12	Cover OK
PTR_SUE_REC_EMPTY	24	Receipt Paper Empty
PTR_SUE_REC_NEAREMPTY	25	Receipt Paper Near Empty
PTR_SUE_REC_PAPEROK	26	Receipt Paper OK
PTR_SUE_IDLE	1001	Printer Idle
PTR_SUE_OFF_LINE	53	Printer off-line
PTR_SUE_ON_LINE	54	Printer on-line
PTR_SUE_BAT_OK	55	Printer battery normal
PTR_SUE_BAT_LOW	56	Printer battery low

5-2-2 Error Event

상수명	값	설명
JPOS_EPTR_COVER_OPEN	201	Cover Open
JPOS_EPTR_REC_EMPTY	203	Paper Empty
JPOS_EPTR_OFF_LINE	217	Printer off-line

5-2-3 OutputComplete Event

인쇄 완료 이벤트를 발생시킵니다. 단, Async 모드로 사용해야 합니다.

5-2-4 Data Event

MSR Track 정보 데이터를 수신 합니다.

5-2-5 DirectIO Event

Direct I/O의 응답 데이터를 수신 합니다.

[Event Example]

```
try
{
	POSPrinter posPrinter = new POSPrinter(this);
	MSR msr = new MSR();
	posPrinter.addErrorListener(this);
	posPrinter.addStatusUpdateListener(this);
	posPrinter.addOutputCompleteListener(this);
	posPrinter.addDirectIOListener(this);
	msr.addDataListener(this);
	posPrinter.setAsyncMode(true);
}
catch(JposException e)
{
	// Error
	e.printStackTrace();
}

@Override
public void outputCompleteOccurred(final OutputCompleteEvent e)
{
	runOnUiThread(new Runnable()
	{
		@Override
		public void run()
		{
			Toast.makeText(MainActivity.this, "complete print", Toast.LENGTH_SHORT).show();
		}
	});
}

@Override
public void dataOccurred(DataEvent arg0)
{
	// TODO Auto-generated method stub
	runOnUiThread(new Runnable()
	{
		@Override
		public void run()
		{
			try
			{
				String strData = new String(msr.getTrack1Data());
				strData += new String(msr.getTrack2Data());
				strData += new String(msr.getTrack3Data());
				Toast.makeText(MainActivity.this, strData, Toast.LENGTH_SHORT).show();
			}
			catch(JposException e)
			{
				Toast.makeText(MainActivity.this, e.getMessage(), Toast.LENGTH_SHORT).show();
			}
		}
	});
}





@Override
```

```
public void directIOOccurred(DirectIOEvent directIOEvent) {
	runOnUiThread(new Runnable()
	{
		@Override
		public void run()
		{
			Toast.makeText(MainActivity.this, new String((byte[]) directIOEvent.getObject(),
	Toast.LENGTH_SHORT).show();
		}
	});
}

@Override
public void errorOccurred(final ErrorEvent arg0)
{
	// TODO Auto-generated method stub
	runOnUiThread(new Runnable()
	{
		@Override
	public void run()
	{
		switch (arg0.getErrorCodeExtended())
		{
		case POSPrinterConst.JPOS_EPTR_COVER_OPEN:
			return "Cover open";
		case POSPrinterConst.JPOS_EPTR_REC_EMPTY:
			return "Paper empty";
		case JposConst.JPOS_SUE_POWER_OFF_OFFLINE:
			return "Power off";
		default:
			return "Unknown";
		}
	}
	});
}

@Override
public void statusUpdateOccurred(final StatusUpdateEvent arg0)
{
	// TODO Auto-generated method stub
	runOnUiThread(new Runnable()
	{
		@Override
		public void run()
		{
			switch (arg0.getStatus())
			{
				case JposConst.JPOS_SUE_POWER_ONLINE:
					return "Power on";
				case JposConst.JPOS_SUE_POWER_OFF_OFFLINE:
					return "Power off";
				case POSPrinterConst.PTR_SUE_COVER_OPEN:
					return "Cover Open";
				case POSPrinterConst.PTR_SUE_COVER_OK:
					return "Cover OK";
				case POSPrinterConst.PTR_SUE_REC_EMPTY:
					return "Receipt Paper Empty";
				case POSPrinterConst.PTR_SUE_REC_NEAREMPTY:
					return "Receipt Paper Near Empty";
				case POSPrinterConst.PTR_SUE_REC_PAPEROK:
					return "Receipt Paper OK";
				case POSPrinterConst.PTR_SUE_IDLE:
					return "Printer Idle";
```

```
                    case POSPrinterConst.PTR_SUE_BAT_LOW:
                        return "Battery-Low";
                    case POSPrinterConst.PTR_SUE_BAT_OK:
                        return "Battery-OK";
                    default:
                        return "Unknown";
                }
            }
        });
}
```

5-3 EscapeSequence

- 인쇄할 텍스트에 폰트, 굵기 등의 옵션을 지정하기 위한 값입니다.
  문자열 형식이며 인쇄 데이터 앞에 추가하여 사용합니다.

정의	설명
String ESCAPE_CHARACTERS = new String(new byte[] {0x1b, 0x7c})	**Escape Characters**
ESCAPE_CHARACTERS + "N"	Normal
ESCAPE_CHARACTERS + "aM"	Font A (12x24)
ESCAPE_CHARACTERS + "bM"	Font B (9x17)
ESCAPE_CHARACTERS + "cM"	Font C (9x24)
ESCAPE_CHARACTERS + "lA"	Left justify
ESCAPE_CHARACTERS + "cA"	Center
ESCAPE_CHARACTERS + "rA"	Right justify
ESCAPE_CHARACTERS + "bC"	Bold
ESCAPE_CHARACTERS + "!bC"	Disabled bold
ESCAPE_CHARACTERS + "uC"	Underline
ESCAPE_CHARACTERS + "!uC"	Disabled underline
ESCAPE_CHARACTERS + "rvC"	Reverse video
ESCAPE_CHARACTERS + "!rvC"	Disabled reverse video
ESCAPE_CHARACTERS + "1C"	Single high and wide
ESCAPE_CHARACTERS + "2C"	Double wide
ESCAPE_CHARACTERS + "3C"	Double high
ESCAPE_CHARACTERS + "4C"	Double high and wide
ESCAPE_CHARACTERS + "1hC"	Scale 1 time horizontally
ESCAPE_CHARACTERS + "2hC"	Scale 2 times horizontally
ESCAPE_CHARACTERS + "3hC"	Scale 3 times horizontally
ESCAPE_CHARACTERS + "4hC"	Scale 4 times horizontally
ESCAPE_CHARACTERS + "5hC"	Scale 5 times horizontally
ESCAPE_CHARACTERS + "6hC"	Scale 6 times horizontally
ESCAPE_CHARACTERS + "7hC"	Scale 7 times horizontally
ESCAPE_CHARACTERS + "8hC"	Scale 8 times horizontally
ESCAPE_CHARACTERS + "1vC"	Scale 1 time vertically
ESCAPE_CHARACTERS + "2vC"	Scale 2 times vertically
ESCAPE_CHARACTERS + "3vC"	Scale 3 times vertically
ESCAPE_CHARACTERS + "4vC"	Scale 4 times vertically
ESCAPE_CHARACTERS + "5vC"	Scale 5 times vertically
ESCAPE_CHARACTERS + "6vC"	Scale 6 times vertically
ESCAPE_CHARACTERS + "7vC"	Scale 7 times vertically
ESCAPE_CHARACTERS + "8vC"	Scale 8 times vertically
ESCAPE_CHARACTERS + "[code]B"	Print NV Image [code] : 0 ~ 255(image code)
ESCAPE_CHARACTERS + "[percentage]fP"	Feed Cut [percentage] : 100 전체절단 [percentage] : 0~90 부분절단

[Example]

```
try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open(logicalDeviceName);
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true)

	String ESCAPE_CHARACTERS = new String(new byte[] {0x1b, 0x7c})

	// 진하게
	String data = ESCAPE_SEQUENCE + "bC" + “Bixolon\n”
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, data);

	// 진하게 해제
	String data = ESCAPE_SEQUENCE + "!bC" + “Bixolon\n”
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, data);

	// NV Image
	String data = ESCAPE_SEQUENCE + "0B";
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, data);

	// Feed cut
	String data = ESCAPE_SEQUENCE + "90fP";
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, data);
}
catch(JposException e)
{
	// Error
	e.printStackTrace();
}
```

5-4 Transaction Print

• Transaction mode 설정 시 사용할 수 있는 값에 대한 정의 입니다.

상수명	값	설명
PTR_TP_TRANSACTION	11	Buffer를 비어있는 상태로 초기화 하고 Transaction Mode 를 시작합니다
PTR_TP_NORMAL	12	Transaction mode 를 종료하고, Buffer에 쌓여 있는 Data 를 출력합니다.

5-5 Alignment

• 정렬을 지정할 때 필요한 값에 대한 정의 입니다.

[For Barcode]

상수명	값	설명
PTR_BC_LEFT	-1	좌측 정렬
PTR_BC_CENTER	-2	중앙 정렬
PTR_BC_RIGHT	-3	우측 정렬

[For Image]

상수명	값	설명
PTR_BM_LEFT	-1	좌측 정렬
PTR_BM_CENTER	-2	중앙 정렬
PTR_BM_RIGHT	-3	우측 정렬

5-6 바코드 타입

• 바코드 출력 시 바코드 지정에 필요한 값에 대한 정의 입니다.

상수명	값	설명
PTR_BCS_UPCA	101	UPCA
PTR_BCS_UPCE	102	UPCE
PTR_BCS_JAN8	103	JAN8
PTR_BCS_EAN8	103	EAN8
PTR_BCS_JAN13	104	JAN13
PTR_BCS_EAN13	104	EAN13
PTR_BCS_TF	105	Standard(ordiscrete) 2 of 5
PTR_BCS_ITF	106	Interleaved 2 of 5
PTR_BCS_Codabar	107	Codabar
PTR_BCS_Code39	108	Code39
PTR_BCS_Code93	109	Code93
PTR_BCS_Code128	110	Code 128 ※ Code128 의 Special Character Special Charaters / Ascii Represntation Code A / {A Code B / {B Code C / {C
PTR_BCS_UPCA_S	111	UPC-A with supplemental barocde
PTR_BCS_UPCE_S	112	UPC-E with supplemental barcode
PTR_BCS_UPCD1	113	UPC-D1
PTR_BCS_UPCD2	114	UPC-D2
PTR_BCS_UPCD3	115	UPC-D3
PTR_BCS_UPCD4	116	UPC-D4
PTR_BCS_UPCD5	117	UPC-D5
PTR_BCS_EAN8_S	118	EAN8 with supplemental barocde
PTR_BCS_EAN13_S	119	EAN13 with supplemental barocde
PTR_BCS_EAN128	120	EAN128
PTR_BCS_OCRA	121	OCR “A”
PTR_BCS_OCRB	122	OCR “B”
PTR_BCS_Code128_Parsed	123	Code 128 with parsing
PTR_BCS_GS1DATABAR	131	GS1 DataBar Omnidirectional
PTR_BCS_GS1DATABAR_E	132	GS1 DataBar Stacked Omnidirectional
PTR_BCS_GS1DATABAR_S	133	GS1 DataBar Expanded
PTR_BCS_GS1DATABAR_E_S	134	GS1 DataBar Expanded Stacked
PTR_BCS_PDF417	201	PDF 417
PTR_BCS_MAXICODE	202	MAXI Code
PTR_BCS_DATAMATRIX	203	Data Matrix
PTR_BCS_QRCODE	204	QR Code
PTR_BCS_UQRCODE	205	Micro QR Code
PTR_BCS_AZTEC	206	Aztec
PTR_BCS_UPDF417	207	Micro PDF 417

5-7 바코드 텍스트 위치

- 텍스트 출력을 지원하는 바코드 일 경우, 바코드 텍스트의 출력 여부 또는 위치를 지정 합니다.

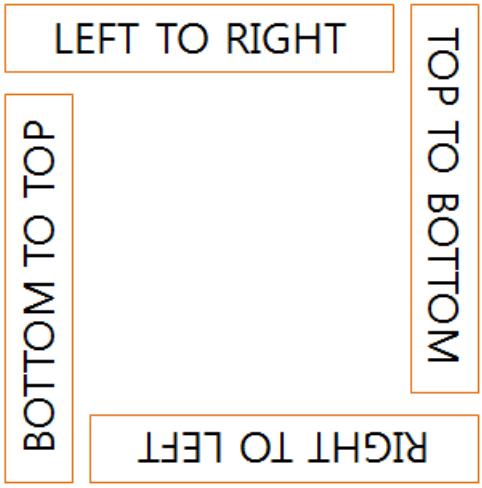
상수명	값	설명
PTR_BC_TEXT_NONE	-11	Text 를 출력하지 않습니다. 바코드만 출력 합니다.
PTR_BC_TEXT_ABOVE	-12	Text 를 바코드의 상단에 출력 합니다.
PTR_BC_TEXT_BELOW	-13	Text 를 바코드의 하단에 출력합니다.

5-8 장치 모델명

상수명	설명
PRODUCT_NAME_SPP_R210	SPP-R210
PRODUCT_NAME_SPP_R215	SPP-R215
PRODUCT_NAME_SPP_R220	SPP-R220
PRODUCT_NAME_SPP_R200II	SPP-R200II
PRODUCT_NAME_SPP_R200III	SPP-R200III
PRODUCT_NAME_SPP_R300	SPP-R300
PRODUCT_NAME_SPP_R310	SPP-R310
PRODUCT_NAME_SPP_R400	SPP-R400
PRODUCT_NAME_SPP_R410	SPP-R410
PRODUCT_NAME_SPP_R418	SPP-R418
PRODUCT_NAME_SRP_350III	SRP-350III
PRODUCT_NAME_SRP_352III	SRP-352III
PRODUCT_NAME_SRP_330II	SRP-330II
PRODUCT_NAME_SRP_332II	SRP-332II
PRODUCT_NAME_SRP_340II	SRP-340II
PRODUCT_NAME_SRP_342II	SRP-342II
PRODUCT_NAME_SRP_350PLUSIII	SRP-350PLUSIII
PRODUCT_NAME_SRP_352PLUSIII	SRP-352PLUSIII
PRODUCT_NAME_SRP_380	SRP-380
PRODUCT_NAME_SRP_382	SRP-382
PRODUCT_NAME_SRP_383	SRP-383
PRODUCT_NAME_SRP_S300	SRP-S300
PRODUCT_NAME_SRP_Q200	SRP-Q200
PRODUCT_NAME_SRP_Q300	SRP-Q300
PRODUCT_NAME_SRP_Q302	SRP-Q302
PRODUCT_NAME_SRP_F310II	SRP-F310II
PRODUCT_NAME_SRP_F312II	SRP-F312II
PRODUCT_NAME_SRP_F313II	SRP-F313II
PRODUCT_NAME_STP_103III	STP-103III
PRODUCT_NAME_SRP_275III	SRP-275III
PRODUCT_NAME_SRP_QE300	SRP-QE300
PRODUCT_NAME_SRP_QE302	SRP-QE302
PRODUCT_NAME_SRP_E300	SRP-E300
PRODUCT_NAME_SRP_E302	SRP-E302
PRODUCT_NAME_BK3_3	BK3-3

5-9 페이지 모드 인쇄 방향

상수명	값	설명
PTR_PD_LEFT_TO_RIGHT	1	왼쪽에서 오른쪽으로 인쇄
PTR_PD_BOTTOM_TO_TOP	2	아래에서 위로 인쇄
PTR_PD_RIGHT_TO_LEFT	3	오른쪽에서 왼쪽으로 인쇄
PTR_PD_TOP_TO_BOTTOM	4	위에서 아래로 인쇄



5-10 MSR 암호화

상수명	값	설명
MSR_DE_NONE	1	데이터 암호화 알고리즘 사용 안함
MSR_DE_3DEA_DUKPT	2	데이터 암호화 알고리즘 사용

5-11 SCR 모드

상수명	값	설명
SC_CMODE_ISO	1	ISO 모드
SC_CMODE_EMV	2	EMV 모드

5-12 Character Set

상수명	값	설명
CS_437_USA_STANDARD_EUROPE	437	PC437
CS_737_GREEK	737	PC737
CS_775_BALTIC	775	PC775
CS_850_MULTILINGUAL	850	PC850
CS_852_LATIN2	852	PC852
CS_855_CYRILLIC	855	PC855
CS_857_TURKISH	857	PC857
CS_858_EURO	858	PC858
CS_860_PORTUGUESE	860	PC860
CS_862_HEBREW_DOS_CODE	862	PC862
CS_863_CANADIAN_FRENCH	863	PC863
CS_864_ARABIC	864	PC864
CS_865_NORDIC	865	PC865
CS_866_CYRILLIC2	866	PC866
CS_928_GREEK	928	PC928
CS_1250_CZECH	1250	WPC1250
CS_1251_CYRILLIC	1251	WPC1251
CS_1252_LATIN1	1252	WPC1252
CS_1253_GREEK	1253	WPC1253
CS_1254_TURKISH	1254	WPC1254
CS_1255_HEBREW_NEW_CODE	1255	WPC1255
CS_1256_ARABIC	1256	WPC1256
CS_1257_BALTIC	1257	WPC1257
CS_1258_VIETNAM	1258	WPC1258
CS_FARSI	7065	FARSI
CS_KATAKANA	7565	KATAKANA
CS_KHMER_CAMBODIA	7572	KHMER
CS_THAI11	8411	THAI11
CS_THAI14	8414	THAI14
CS_THAI16	8416	THAI16
CS_THAI18	8418	THAI18
CS_THAI42	8442	THAI42
CS_KS5601	5601	KS5601
CS_BIG5	6605	BIG5
CS_GB2312	2312	GB2312
CS_SHIFT_JIS	8374	SHIFT-JIS
CS_TCVN_3_1	3031	TCVN-3(1)
CS_TCVN_3_2	3032	TCVN-3(2)

6. 각 클래스별 기능 정리

6-1 BXLConfigLoader Class

- 연결하고자 하는 장치 설정 정보를 저장하기 위한 Class 입니다. 설정 정보는 BXLConfigLoader Class를 통해 장치 정보를 관리 합니다. 설정 정보에는 장치 이름, 제품명, 인터페이스 등이 있으며 해당 정보가 정상적으로 저장되어 있지 않으면 장치에 연결할 수 없습니다. Open 함수를 호출하기 전에 이 클래스를 호출하여 설정 정보를 저장해야 합니다.

주의	BXLConfigLoader를 통해 저장되지 않은 장치는 연결 할 수 없습니다.

6-1-1 openFile()

기존 저장된 설정 파일을 Open 합니다.

[**Syntax**]
void openFile() throws Exception

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;

try
{
	BXLConfigLoader bxlConfigLoader = new BXLConfigLoader(this);
	bxlConfigLoader.openFile();
}
catch(Exception e)
{
	// Error
	e.printStackTrace();
}
```

6-1-2 newFile()

새로운 설정 저장 파일을 생성 합니다. 보통 openFile 예외 발생 시 수행 합니다.

[**Syntax**]
void newFile() throws Exception

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;

try
{
	BXLConfigLoader bxlConfigLoader = new BXLConfigLoader(this);
	bxlConfigLoader.openFile();
}
catch(Exception e)
{
	e.printStackTrace();
	bxlConfigLoader.newFile();
}
```

6-1-3 getEntries()

저장된 설정 정보를 얻어옵니다.

[**Syntax**]
List<JposEntry> getEntries() throws Exception

[**Return Values**]

값	설명
JposEntry의 List 컨테이너	객체의 리스트 배열

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;

try
{
	for (Object entry : bxlConfigLoader.getEntries())
	{
		JposEntry jposEntry = (JposEntry)entry;
		String strLogicalname = jposEntry.getLogicalName();
	}
}
catch(Exception e)
{
	e.printStackTrace();
}
```

6-1-4 addEntry()

장치 연결 정보를 추가 합니다.

[Syntax]
void addEntry(String logicalName, int deviceCategory, String productName,
int deviceBus, String address) throws IllegalArgumentException

[Parameters]
• String logicalName : 장치의 논리적 이름(별칭)을 입력합니다.
• int deviceCategory : 장치의 종류를 선택 합니다.

상수명	값	설명
DEVICE_CATEGORY_CASH_DRAWER	0	Cash Drawer
DEVICE_CATEGORY_MSR	1	MSR
DEVICE_CATEGORY_POS_PRINTER	2	POS Printer
DEVICE_CATEGORY_SMART_CARD_RW	3	SCR
DEVICE_CATEGORY_LOCAL_SMART_CARD_RW	4	SCR(SPP-R215 전용)

• String productName : 장치 모델명(“5-8 장치 모델명” 참조)
• int deviceBus : 인터페이스 타입을 선택 합니다.

상수명	값	설명
DEVICE_BUS_BLUETOOTH	0	Bluetooth
DEVICE_BUS_ETHERNET	1	Ethernet
DEVICE_BUS_USB	2	USB
DEVICE_BUS_WIFI	3	WiFi
DEVICE_BUS_WIFI_DIRECT	4	WiFi-Driect
DEVICE_BUS_BLUETOOTH_LE	5	Bluetooth Low Energy

• String address : 장치의 MAC 또는 IP Address를 입력합니다.
(BT : MAC Address, Netwrok : IP Address)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;

try
{
    BXLConfigLoader bxlConfigLoader = new BXLConfigLoader(this);
    bxlConfigLoader.addEntry("SPP-R200III",
                             BXLConfigLoader.DEVICE_CATEGORY_POS_PRINTER,
                             BXLConfigLoader.PRODUCT_NAME_SPP_R200III,
                             BXLConfigLoader.DEVICE_BUS_BLUETOOTH,
                             "74:F0:7D:E4:11:AF");
}
catch(Exception e)
{
    e.printStackTrace();
}
```

6-1-5 removeEntry()

저장된 설정 정보를 삭제 합니다.

[**Syntax**]
boolean removeEntry(String logicalName)

[**Parameters**]
• String logicalName : 장치의 논리적 이름(별칭)을 입력합니다.

[**Return Values**]

값	설명
true	성공 시 리턴 됩니다.
false	실패 시 리턴 됩니다.

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;

try
{
	for (Object entry : bxlConfigLoader.getEntries())
	{
		JposEntry jposEntry = (JposEntry)entry;
		String strLogicalname = jposEntry.getLogicalName();

		bxlConfigLoader.removeEntry(strLogicalname);
	}
}
catch(Exception e)
{
	e.printStackTrace();
}
```

6-1-6 saveFile()

addEntry, removeEntry를 통해 변경된 정보를 저장 합니다.

[**Syntax**]
void saveFile()

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;

try
{
	for (Object entry : bxlConfigLoader.getEntries())
	{
		JposEntry jposEntry = (JposEntry)entry;
		String strLogicalname = jposEntry.getLogicalName();

		bxlConfigLoader.removeEntry(strLogicalname);
	}

	bxlConfigLoader.addEntry("SPP-R200III",
					BXLConfigLoader.DEVICE_CATEGORY_POS_PRINTER,
					BXLConfigLoader.PRODUCT_NAME_SPP_R200III,
					BXLConfigLoader.DEVICE_BUS_BLUETOOTH,
					"74:F0:7D:E4:11:AF");

	bxlConfigLoader.saveFile();
}
catch(Exception e)
{
	e.printStackTrace();
}
```

주의	addEntry 함수를 통해 추가된 장치는 반드시 saveFile 함수를 통해 저장해야 합니다.

6-2 POSPrinter Class

• POS Printer 제어를 위한 Class 입니다. 해당 Class를 사용하여 프린터 연결 및 해제와 인쇄 작업을 수행 합니다. 특정 기능 수행 중 에러 발생 시 JposException을 발생시킵니다. (“5-1 JposException” 참조)

6-2-1 open()

프린터 클래스 사용을 시작하며 메모리 할당 등의 초기화 작업이 포함되어있습니다.
Claim 이상의 Method 호출을 위해서 반드시 선행 되어야 합니다.
BXLConfigLoader Class를 통해 저장되지 않은 장치는 open되지 않습니다.

[**Syntax**]
void open(String logicalDeviceName) throws JposException

[**Parameters**]
• String logicalDeviceName : Open 하려는 장치의 이름을 입력 합니다.

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open(“SPP-R200III”);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-2 claim()

Device 정보에 포함되어있는 Port 를 실제로 Open 하기 위해 시도하며, 메모리 할당 초기화 등의 초기화 작업이 일부 포함되어있습니다.
Device 사용을 활성화하기 위해서 반드시 선행되어야 합니다.

[**Syntax**]
void claim(int timeout) throws JposException

[**Parameters**]
• int timeout : 이 파라미터에서 지정한 시간동안 Port open을 시도합니다.

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-3 setDeviceEnabled()

장치 사용 여부를 포함하고 있습니다.
DeviceEnabled 의 값이 비활성화 되어 있으면 기능 이용이 불가능 할 수 있습니다.

[**Syntax**]
void setDeviceEnabled(boolean deviceEnabled) throws JposException

[**Parameters**]
• boolean deviceEnabled : 장치 활성화 상태를 입력 합니다..

값	설명
true	활성화
false	비활성화

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-4 release()

Claim 된 Device의 Port 사용을 물리적으로 종료합니다.
메모리 해제 등의 작업이 수행될 수 있습니다

[**Syntax**]
void release() throws JposException

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	posPrinter.release();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-5 close()

Open 된 Device의 사용을 종료합니다.
메모리 해제 등의 작업이 일부 수행될 수 있습니다.

[**Syntax**]
void close() throws JposException

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	posPrinter.close();
	posPrinter.setDeviceEnabled(false);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-6 checkHealth()

장치가 정상적으로 동작하는 상태인지 확인 합니다.
Open / Claim / DeviceEnabled 가 정상적으로 수행된 상태일 때 사용 가능 합니다.

[**Syntax**]
void checkHealth(int level) throws JposException

[**Parameters**]
• int level : 고정값 JposConst.JPOS_CH_INTERNAL

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.checkHealth(JposConst.JPOS_CH_INTERNAL);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-7 setAsyncMode()

비동기 모드 사용 여부를 포함하고 있습니다.
비동기 모드가 true일 경우 관련 메소드가 비동기 모드로 동작합니다. False일 경우 관련 메소드는 동기 모드로 동작 합니다.

[**Syntax**]
void setAsyncMode(boolean asyncMode) throws JposException

[**Parameters**]
• boolean asyncMode : 비동기 모드 사용 여부를 선택 합니다.

값	설명
true	비동기 모드
false	동기 모드

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.setAsyncMode(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-8 setCharacterSet()

프린터의 Character set을 설정 합니다.

[**Syntax**]
void setCharacterSet(int characterSet) throws JposException

[**Parameters**]
• int characterSet : 프린터에 설정할 Character set을 선택 합니다.
(“5-12 Character Set” 참조)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open(“SPP-R200III”);
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.setCharacterSet(BXLConst.CS_437_USA_STANDARD_EUROPE);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-9 setCharacterEncoding()

프린터로 전송할 데이터의 인코딩을 설정 합니다.

[**Syntax**]
void setCharacterEncoding(int characterEncoding) throws JposException

[**Parameters**]
• int characterEncoding : 데이터 인코딩 타입을 선택 합니다.

상수명	값	설명
CE_ASCII	0	ASCII(default)
CE_UTF8	1	UTF-8

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.setCharacterEncoding(BXLConst.CE_ASCII);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-10 cutPaper()

Auto Cutter 가 내장된 모델의 경우 종이를 절단합니다.

[**Syntax**]
void cutPaper(int percentage) throws JposException

[**Parameters**]
• int percentage : Full cut/Partial cut 을 선택 합니다.
커터가 달린 모델에 한해 동작 합니다.

값	설명
100	Full cut
90	Partial cut

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.cutPaper(100);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-11 printBarCode()

바코드를 인쇄 합니다.

[**Syntax**]
void printBarCode(int station, String data, int symbology, int height, int width, int alignment, int textPosition) throws JposException

[**Parameters**]
- int station : 고정값 PTR_S_RECEIPT
- String data : 바코드에 포함할 Data, 바코드 타입 별 허용되는 Data가 다를 수 있습니다.
- int symbology : Barcode 종류를 선택 합니다.("5-6 바코드 타입" 참조)
- int height : Barcode 의 높이를 지정합니다.
- int width : Barcode 의 너비를 지정합니다.
- int alignment : 바코드 정렬을 선택합니다. ("5-5 Alignment" 참조)
- int textPosition : 바코드와 함께 출력 될 Text의 Postion 을 결정합니다. ("5-7 바코드 텍스트 위치" 참조)

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.printBarCode(POSPrinterConst.PTR_S_RECEIPT,
	                        "123456789",
	                        POSPrinterConst.PTR_BCS_QRCODE,
	                        8,
	                        8,
	                        POSPrinterConst.PTR_BC_CENTER,
	                        POSPrinterConst.PTR_BC_TEXT_BELOW);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-12 printBitmap()

이미지를 인쇄 합니다.(파일 인쇄)

[Syntax]
void printBitmap(int station, String fileName, int width, int alignment) throws JposException

[Parameters]
• int station : 이미지 인쇄 옵션을 설정 합니다.(총 4바이트로 구성)

Index	설명
첫번째 바이트	고정값 : PTR_S_RECEIPT
두번째 바이트	밝기 : 0 ~ 100
세번째 바이트	압축 알고리즘(RLE) 사용여부(0x01 : RLE)
네번째 바이트	Reserved(0x00)

• String filename : 이미지 파일의 경로를 지정합니다.
• int width : 이미지 너비를 지정합니다.
• int alignment : 이미지 정렬을 선택합니다. (“5-5 Alignment” 참조)

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	ByteBuffer buffer = ByteBuffer.allocate(4);
	buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
	buffer.put((byte) 80); // brightness
	buffer.put((byte) 0x01); // compress
	buffer.put((byte) 0x00);

	posPrinter.printBitmap(buffer.getInt(0),
				imagePath,
				posPrinter.getRecLineWidth(),
				POSPrinterConst.PTR_BM_LEFT);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-13 printBitmap()

이미지를 인쇄 합니다.(Bitmap 데이터)

[Syntax]
void printBitmap(int station, Bitmap bitmap, int width, int alignment) throws JposException

[Parameters]
• int station : 이미지 인쇄 옵션을 설정 합니다.(총 4바이트로 구성)

Index	설명
첫번째 바이트	고정값 : PTR_S_RECEIPT
두번째 바이트	밝기 : 0 ~ 100
세번째 바이트	압축 알고리즘(RLE) 사용여부(0x01 : RLE)
네번째 바이트	Reserved(0x00)

• Bitmap bitmap : 비트맵 형식의 이미지 데이터를 입력합니다.
• int width : 이미지 너비를 지정합니다.
• int alignment : 이미지 정렬을 선택합니다. (“5-5 Alignment” 참조)

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	ByteBuffer buffer = ByteBuffer.allocate(4);
	buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
	buffer.put((byte) 80); // brightness
	buffer.put((byte) 0x01); // compress
	buffer.put((byte) 0x00);

	posPrinter.printBitmap(buffer.getInt(0),
				BitmapData,
				posPrinter.getRecLineWidth(),
				POSPrinterConst.PTR_BM_LEFT);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-14 printNormal()

텍스트를 인쇄 합니다.

[**Syntax**]
void printNormal(int station, String data) throws JposException

[**Parameters**]
• int station : 고정값 PTR_S_RECEIPT
• String data : 출력할 Data를 지정합니다. 출력 가능한 문자와 escape sequences, carriage returns, line feeds Data가 허용됩니다.

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, "Print Data\n");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-15 setPageModePrintArea()

페이지 모드 영역을 지정 합니다.

[**Syntax**]
void setPageModePrintArea(String area) throws JposException

[**Parameters**]
• String area : 인쇄 영역을 지정 합니다.
"영역의 x 좌표, 영역의 y 좌표, 영역의 가로, 영역의 세로"
ex) "0, 0, 576, 1600"

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.setPageModePrintArea("0, 0, 576, 1600");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-16 setPageModePrintDirection()

페이지 모드 인쇄 방향을 지정 합니다.

[**Syntax**]
void setPageModePrintDirection(int direction) throws JposException

[**Parameters**]
• int direction : 인쇄 방향을 지정 합니다.(“5-9 페이지 모드 인쇄 방향” 참조)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open(“SPP-R200III”);
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.setPageModePrintArea(“0, 0, 576, 1600”);
	posPrinter.setPageModePrintDirection(POSPrinterConst.PTR_PD_LEFT_TO_RIGHT);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-17 pageModePrint()

프린터를 페이지 모드로 설정 합니다.

[Syntax]
void pageModePrint(int control) throws JposException

[Parameters]
• int control : 페이지 모드를 지정 합니다.

상수명	값	설명
PTR_PM_PAGE_MODE	1	페이지 모드 활성화
PTR_PM_NORMAL	2	일반 모드로 변경되며 페이지 모드 버퍼에 저장된 데이터가 인쇄 됩니다.

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.setPageModePrintArea("0, 0, 576, 1600");
	posPrinter.setPageModePrintDirection(POSPrinterConst.PTR_PD_LEFT_TO_RIGHT);
	posPrinter.setPageModePrint(POSPrinterConst.PTR_PM_PAGE_MODE);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-18 setPageModeHorizontalPosition()

인쇄 시작 위치(가로)를 지정 합니다.

[**Syntax**]
void setPageModeHorizontalPosition(int position) throws JposException

[**Parameters**]
• int position : 인쇄 시작 위치(가로)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	ByteBuffer buffer = ByteBuffer.allocate(4);
	buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
	buffer.put((byte) 80); // brightness
	buffer.put((byte) 0x01); // compress
	buffer.put((byte) 0x00);

	// 인쇄 영역 지정
	posPrinter.setPageModePrintArea("0, 0, 576, 1600");
	// 인쇄 방향 설정
	posPrinter.setPageModePrintDirection(POSPrinterConst.PTR_PD_LEFT_TO_RIGHT);
	// 프린터 페이지 모드로 변환
	posPrinter.setPageModePrint(POSPrinterConst.PTR_PM_PAGE_MODE);

	// 가로/세로 인쇄 위치 지정
	posPrinter.setPageModeHorizontalPosition(0);
	posPrinter.setPageModeVerticalPosition(0);
	// 인쇄할 데이터 전송(이미지)
	posPrinter.printBitmap(buffer.getInt(0), imagePath, 384, POSPrinterConst.PTR_BM_LEFT);

	// 가로/세로 인쇄 위치 지정
	posPrinter.setPageModeHorizontalPosition(100);
	posPrinter.setPageModeVerticalPosition(100);
	// 인쇄할 데이터 전송(텍스트)
	posPrinter.printNormar(POSPrinterConst.PTR_S_RECEIPT, "Print Data\n");

	// 인쇄 시작
	posPrinter.setPageModePrint(POSPrinterConst.PTR_PM_PAGE_NORMAL);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-19 setPageModeVerticalPosition()

인쇄 시작 위치(세로)를 지정 합니다.

[**Syntax**]
void setPageModeVerticalPosition(int position) throws JposException

[**Parameters**]
• int position : 인쇄 시작 위치(세로)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	ByteBuffer buffer = ByteBuffer.allocate(4);
	buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
	buffer.put((byte) 80); // brightness
	buffer.put((byte) 0x01); // compress
	buffer.put((byte) 0x00);

	// 인쇄 영역 지정
	posPrinter.setPageModePrintArea("0, 0, 576, 1600");
	// 인쇄 방향 설정
	posPrinter.setPageModePrintDirection(POSPrinterConst.PTR_PD_LEFT_TO_RIGHT);
	// 프린터 페이지 모드로 변환
	posPrinter.setPageModePrint(POSPrinterConst.PTR_PM_PAGE_MODE);

	// 가로/세로 인쇄 위치 지정
	posPrinter.setPageModeHorizontalPosition(0);
	posPrinter.setPageModeVerticalPosition(0);
	// 인쇄할 데이터 전송(이미지)
	posPrinter.printBitmap(buffer.getInt(0), imagePath, 384, POSPrinterConst.PTR_BM_LEFT);

	// 가로/세로 인쇄 위치 지정
	posPrinter.setPageModeHorizontalPosition(100);
	posPrinter.setPageModeVerticalPosition(100);
	// 인쇄할 데이터 전송(텍스트)
	posPrinter.printNormar(POSPrinterConst.PTR_S_RECEIPT, "Print Data\n");

	// 인쇄 시작
	posPrinter.setPageModePrint(POSPrinterConst.PTR_PM_PAGE_NORMAL);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-20 transactionPrint()

Transaction Mode를 사용하여 인쇄 합니다.

[**Syntax**]
public void transactionPrint(int station, int control) throws JposException

[**Parameters**]
• int station : 고정값 PTR_S_RECEIPT
• int control : Transaction Mode(“5-4 Transaction Print” 참조)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open(“SPP-R200III”);
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	// Transaction mode 시작
	posPrinter.transactionPrint(POSPrinterConst.PTR_TP_TRANSACTION);

	// 인쇄할 데이터 전송(이미지)
	posPrinter.printBitmap(buffer.getInt(0), imagePath, 384, POSPrinterConst.PTR_BM_LEFT);
	// 인쇄할 데이터 전송(텍스트)
	posPrinter.printNormar(POSPrinterConst.PTR_S_RECEIPT, “Print Data\n”);

	// Transaction mode 종료(인쇄 시작)
	posPrinter.transactionPrint(POSPrinterConst.PTR_TP_NORMAL);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-21 displayString()

BCD-3000에 텍스트를 출력 합니다.
단, BCD-3000은 SRP-Q300 DK port에 연결되어 있어야 합니다.

[**Syntax**]
public void displayString(String data) throws JposException

[**Parameters**]
• String data : BCD-3000에 출력할 텍스트 데이터

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SRP-Q300");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.displayString("BIXOLON Customer Display 3000");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-22 cleanScreen()

BCD-3000 화면을 클리어 합니다.
단, BCD-3000은 SRP-Q300 DK port에 연결되어 있어야 합니다.

[**Syntax**]
public void cleanScreen() throws JposException

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SRP-Q300");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.displayString("BIXOLON Customer Display 3000");
	posPrinter.cleanScreen();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-23 storeImageFile()

BCD-3000 이미지 버퍼에 이미지 데이터를 저장 합니다.
단, BCD-3000은 SRP-Q300 DK port에 연결되어 있어야 합니다.

[**Syntax**]
public void storeImageFile(String filename, int width, int height, int imageNumber) throws JposException

[**Parameters**]
• String filename : 이미지 파일의 경로를 지정합니다.
• int width : 이미지 너비를 지정합니다.(1 ~ 160)
• int height : 이미지 높이를 지정합니다.(1 ~ 32)
• int imageNumber : 저장할 이미지 데이터 번호를 지정합니다.(1 ~ 5)

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SRP-Q300");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.storeImageFile(imagePath, 160, 32, 1);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-24 displayImage()

BCD-3000 이미지 버퍼에 저장된 이미지를 출력 합니다.
단, BCD-3000은 SRP-Q300 DK port에 연결되어 있어야 합니다.

[**Syntax**]
public void displayImage(int imageNumber, int xPos, int yPos) throws JposException

[**Parameters**]
• int imageNumber : 출력할 이미지 데이터 번호를 지정합니다.(1 ~ 5)
• int xPos : 이미지를 출력할 X 좌표를 입력 합니다.(0 ~ 159)
• int yPos : 이미지를 출력할 Y 좌표를 입력 합니다.(0 ~ 31)
※ 이미지가 출력할 영역을 넘어갈 경우 이미지는 Display 되지 않습니다.

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SRP-Q300");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.storeImageFile(imagePath, 160, 32, 1);
	posPrinter.displayImage(1, 0, 0);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-25 clearImage()

BCD-3000 이미지 버퍼에 저장된 이미지를 삭제 합니다.
단, BCD-3000은 SRP-Q300 DK port에 연결되어 있어야 합니다.

[**Syntax**]
public void clearImage(boolean isAll, int imageNumber) throws JposException

[**Parameters**]
• boolean isAll : 이미지 버퍼 클리어 모드를 선택 합니다.

값	설명
true	이미지 버퍼의 모든 이미지를 삭제합니다.
false	지정된 번호의 이미지만 삭제 합니다. imageNumber에 삭제할 이미지 번호를 입력해야 합니다.

• int imageNumber : 삭제할 이미지 데이터 번호를 지정합니다. (1 ~ 5)

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SRP-Q300");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.storeImageFile(imagePath, 160, 32, 1);
	posPrinter.displayImage(1, 0, 0);
	posPrinter.clearImage(false, 1);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-26 directIO()

사용자 정의 데이터를 프린터에 전송합니다.
데이터에 따른 응답값은 directIOOccurred Event로 전달 됩니다.

[**Syntax**]
public void directIO(int command, int[] data, Object object) throws JposException

[**Parameters**]
• int command : direct I/O command 종류를 선택 합니다.

값	설명
1	사용자 정의 데이터 전송.
2	배터리 상태 체크.
3	Power off time(모바일 프린터만 사용 가능) : 0 ~ 90(초)

• int[] data : Power off time을 지정 합니다.(commad = 3)
• Object object : 사용자 정의 명령어를 입력 합니다.

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.addDirectIOListener(this);
	posPrinter.open("SRP-Q300");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	byte[] data = new byte{0x1d, 0x49, 0x43};
	posPrinter.directIO(1, null, data);
}
catch(JposException e)
{
	e.printStackTrace();
}

@Override
public void directIOOccurred(DirectIOEvent directIOEvent) {
	runOnUiThread(new Runnable()
	{
		@Override
		public void run()
		{
			Toast.makeText(MainActivity.this, new String((byte[]) directIOEvent.getObject(),
	Toast.LENGTH_SHORT).show();
		}
	});
}
```

6-2-27 markFeed()

다음 인쇄 위치까지 용지를 Feeding 합니다.

[**Syntax**]
public void markFeed(int type) throws JposException

[**Parameters**]
• int type : Mark type을 지정 합니다.(고정값 : 0)

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R310");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.markFeed(0);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-28 setBitmap()

프린터의 비휘발성 메모리 영역에 이미지를 저장합니다.

[Syntax]
void setBitmap(int bitmapNumber, int station, String fileName, int width, int alignment) throws JposException

[Parameters]
• int bitmapNumber : 이미지를 저장할 메모리 영역의 주소(0 ~ 255)
• int station : 이미지 인쇄 옵션을 설정 합니다.(총 4바이트로 구성)

Index	설명
첫번째 바이트	고정값 : PTR_S_RECEIPT
두번째 바이트	밝기 : 0 ~ 100
세번째 바이트	Reserved(0x00)
네번째 바이트	Reserved(0x00)

• String filename : 이미지 파일의 경로를 지정합니다.
• int width : 이미지 너비를 지정합니다.
• int alignment : 고정값 : -1

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	ByteBuffer buffer = ByteBuffer.allocate(4);
	buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
	buffer.put((byte) 80); // brightness
	buffer.put((byte) 0x00); // Reserved
	buffer.put((byte) 0x00); // Reserved

	// Define NV image : Memory code 0
	posPrinter.setBitmap(0,
				buffer.getInt(0),
				imagePath,
				posPrinter.getRecLineWidth(),
				-1);

	// Print NV image : Memory code 0
	String data = ESCAPE_SEQUENCE + "0B";
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, data);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-2-29 printSvg()

SVG 파일를 인쇄 합니다.

[Syntax]
void printSvg(int station, String fileName, int width, int alignment, int rotate) throws JposException

[Parameters]
• int station : SVG 인쇄 옵션을 설정 합니다.(총 4바이트로 구성)

Index	설명
첫번째 바이트	고정값 : PTR_S_RECEIPT
두번째 바이트	밝기 : 0 ~ 100
세번째 바이트	압축 알고리즘(RLE) 사용여부(0x01 : RLE)
네번째 바이트	Reserved(0x00)

• String filename : SVG 파일의 경로를 지정합니다.
• int width : SVG 인쇄 너비를 지정합니다.
• int alignment : SVG 인쇄 정렬을 선택합니다. (“5-5 Alignment” 참조)
• int rotate : SVG 파일 회전을 선택합니다.

값	설명
0	회전 없음
90	90° 회전
180	180° 회전
270	270° 회전

[Example]
```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
    POSPrinter posPrinter = new POSPrinter(this);
    posPrinter.open("SPP-R410");
    posPrinter.claim(5000);
    posPrinter.setDeviceEnabled(true);

    ByteBuffer buffer = ByteBuffer.allocate(4);
    buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
    buffer.put((byte) 50); // brightness
    buffer.put((byte) 0x01); // compress
    buffer.put((byte) 0x00);

    posPrinter.printSvg(buffer.getInt(0),
                        imagePath,
                        posPrinter.getRecLineWidth(),
                        POSPrinterConst.PTR_BM_LEFT,
                        90);
}
catch(JposException e)
{
    e.printStackTrace();
}
```

6-2-30 updateFrimware()

프린터 펌웨어를 업데이트 합니다. 펌웨어 업데이트 이 후 프린터를 재부팅해야 합니다.

[**Syntax**]
void updateFirmware(String firmwareFileName) throws JposException

[**Parameters**]
• String firmwareFileName : 프린터로 전송할 펌웨어 이진 파일의 절대 경로.

[Example]

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;
import com.bxl.BXLConst;

try
{
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	posPrinter.updateFirmware("fw.fls");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3 MSR Class

- 프린터의 MSR(Magnetic Stripe Reader) 제어를 위한 Class 입니다. 해당 Class를 사용하여 MSR로부터 Card Track 정보를 얻어올 수 있습니다. 특정 기능 수행 중 에러 발생 시 JposException을 발생시킵니다. (“5-1 JposException” 참조)

6-3-1 open()

MSR 클래스 사용을 시작하며 메모리 할당 등의 초기화 작업이 포함되어있습니다.
Claim 이상의 Method 호출을 위해서 반드시 선행 되어야 합니다.
BXLConfigLoader Class를 통해 저장되지 않은 장치는 open되지 않습니다.

[**Syntax**]
void open(String logicalDeviceName) throws JposException

[**Parameters**]
- String logicalDeviceName : Open 하려는 장치 이름(MSR)

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-2 claim()

Device 정보에 포함되어있는 Port 를 실제로 Open 하기 위해 시도 하며, 메모리 할당 초기화 등의 초기화 작업이 일부 포함되어있습니다.
Device 사용을 활성화 하기 위해서 반드시 선행 되어야 합니다.

[**Syntax**]
void claim(int timeout) throws JposException

[**Parameters**]
• int timeout : 이 파라미터에서 지정한 시간동안 Port open을 시도합니다.

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-3 setDeviceEnabled()

장치 사용 여부를 포함하고 있습니다.
DeviceEnabled 의 값이 비활성화 되어 있으면 기능 이용이 불가능 할 수 있습니다.

[**Syntax**]
void setDeviceEnabled(boolean deviceEnabled) throws JposException

[**Parameters**]
• boolean deviceEnabled : 장치 활성화 상태를 입력 합니다..

값	설명
true	활성화
false	비활성화

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
	msr.setDeviceEnabled(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-4 release()

Claim 된 Device의 Port 사용을 물리적으로 종료 합니다.
메모리 해제 등의 작업이 수행 될 수 있습니다

[**Syntax**]
void release() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	msr.release();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-5 close()

Open 된 Device의 사용을 종료 합니다.
메모리 해제 등의 작업이 일부 수행 될 수 있습니다.

[**Syntax**]
void close() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	msr.close();
	msr.setDeviceEnabled(false);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-6 setAutoDisable()

MSR 리딩 후 자동으로 MSR 장치를 비활성화 합니다.
AutoDisable가 true일 경우 MSR 데이터 수신 후 DeviceEnabled를 false로 변경 합니다.

[**Syntax**]
void setAutoDisable(boolean autoDisable) throws JposException

[**Parameters**]
• boolean autoDisable : Auto Disable 여부를 지정 합니다.

값	설명
true	활성화
false	비활성화

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
	msr.setDeviceEnabled(true);

	msr.setAutoDisable(true);
	// msr.setAutoDisable(false);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-7 setDataEventEnabled()

MSR 리딩 후 Track 정보를 Data Event를 통해 수신할지 여부를 지정합니다.

[**Syntax**]
void setDataEventEnabled(boolean dataEventEnabled) throws JposException

[**Parameters**]
• boolean dataEventEnabled : Data Event 사용 여부를 지정 합니다.

값	설명
true	Event 사용(“5-2.4 Data Event” 참조)
false	Event 미사용

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open(“MSR”);
	msr.claim(3000);
	msr.setDeviceEnabled(true);

	msr.addDataListener(this);
	msr.setDataEventEnabled(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-8 setDataEncryptionAlgorithm()

MSR Track 데이터의 암호화 여부를 지정합니다.

[**Syntax**]
void setDataEncryptionAlgorithm(int encryptAlgorithm) throws JposException

[**Parameters**]
• int encryptAlgorithm : Track 암호화 사용 여부를 지정 합니다.("5-10 MSR 암호화" 참조)

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
	msr.setDeviceEnabled(true);

	msr. setDataEncryptionAlgorithm(MSRConst.MSR_DE_NONE);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-9 getTrack1Data()

가장 최근에 읽은 MSR Card의 Track1 Data 를 가져 옵니다.

[**Syntax**]
byte[] getTrack1Data() throws JposException

[**Return Values**]
※ MSR Card의 Track1 Data

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
	msr.setDeviceEnabled(true);

	byte[] track1 = msr. getTrack1Data();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-10 getTrack2Data()

가장 최근에 읽은 MSR Card의 Track2 Data 를 가져 옵니다.

[**Syntax**]
byte[] getTrack2Data() throws JposException

[**Return Values**]
※ MSR Card의 Track2 Data

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
	msr.setDeviceEnabled(true);

	byte[] track2 = msr. getTrack2Data();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-3-11 getTrack3Data()

가장 최근에 읽은 MSR Card의 Track3 Data 를 가져 옵니다.

[**Syntax**]
byte[] getTrack3Data() throws JposException

[**Return Values**]
※ MSR Card의 Track3 Data

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.MSR;
import jpos.MSRConst;
import jpos.events.DataEvent;
import jpos.events.DataListener;

try
{
	MSR msr = new MSR();
	msr.open("MSR");
	msr.claim(3000);
	msr.setDeviceEnabled(true);

	byte[] track3 = msr. getTrack3Data();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4 SmartCardRW Class

• 프린터의 SCR(Smart Card Reader) 제어를 위한 Class 입니다. 특정 기능 수행 중 에러 발생 시 JposException을 발생시킵니다. (“5-1 JposException” 참조)

6-4-1 open()

SmartCardRW 클래스 사용을 시작하며 메모리 할당 등의 초기화 작업이 포함되어있습니다.
Claim 이상의 Method 호출을 위해서 반드시 선행 되어야 합니다.
BXLConfigLoader Class를 통해 저장되지 않은 장치는 open되지 않습니다.

[**Syntax**]
void open(String logicalDeviceName) throws JposException

[**Parameters**]
• String logicalDeviceName : Open 하려는 장치 이름(SmartCardRW)

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open(“SmartCardRW”);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-2 claim()

Device 정보에 포함되어있는 Port 를 실제로 Open 하기 위해 시도 하며, 메모리 할당 초기화 등의 초기화 작업이 일부 포함되어있습니다.
Device 사용을 활성화 하기 위해서 반드시 선행 되어야 합니다.

[**Syntax**]
void claim(int timeout) throws JposException

[**Parameters**]
• int timeout : 이 파라미터에서 지정한 시간동안 Port open을 시도합니다.

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(3000);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-3 setDeviceEnabled()

장치 사용 여부를 포함하고 있습니다.
DeviceEnabled 의 값이 비활성화 되어 있으면 기능 이용이 불가능 할 수 있습니다.

[**Syntax**]
void setDeviceEnabled(boolean deviceEnabled) throws JposException

[**Parameters**]
• boolean deviceEnabled : 장치 활성화 상태를 입력 합니다..

값	설명
true	활성화
false	비활성화

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(3000);
	SmartCardRW.setDeviceEnabled(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-4 release()

Claim 된 Device의 Port 사용을 물리적으로 종료 합니다.
메모리 해제 등의 작업이 수행 될 수 있습니다

[**Syntax**]
void release() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	smartCardRW.release();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-5 close()

Open 된 Device의 사용을 종료 합니다.
메모리 해제 등의 작업이 일부 수행 될 수 있습니다.

[**Syntax**]
void close() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	smartCardRW.close();
	smartCardRW.setDeviceEnabled(false);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-6 setSCSlot()

통신을 위한 Card slot을 선택 합니다.

[Syntax]
void setSCSlot() throws JposException

[Parameters]
• int scSlot : 카드 슬롯을 선택 합니다.

값	설명
0x1000	Smart Card
0x0100	SAM1
0x0010	SAM2

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(3000);
	SmartCardRW.setDeviceEnabled(true);

	smartCardRW.setSCSlot(0x01 << (Integer.SIZE - 1)); // Smart Card
	smartCardRW.setSCSlot(0x01 << (Integer.SIZE - 2)); // SAM1
	smartCardRW.setSCSlot(0x01 << (Integer.SIZE - 3)); // SAM2
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-7 setIsoEmvMode()

ISO/EMV 모드를 선택 합니다.

[**Syntax**]
void setIsoEmvMode(int isoEmvMode) throws JposException

[**Parameters**]
• int isoEmvMode : 모드를 선택 합니다.(“5-11 SCR 모드” 참조)

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open(“SmartCardRW”);
	smartCardRW.claim(3000);
	SmartCardRW.setDeviceEnabled(true);

	smartCardRW.setIsoEmvMode(SmartCardRWConst.SC_CMODE_EMV); // EMV Mode
	smartCardRW.setIsoEmvMode(SmartCardRWConst.SC_CMODE_ISO); // ISO Mode
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-8 beginInsertion()

지정한 시간 동안 Smart Card가 삽입되었는지 체크 합니다.
해당 함수 호출 후 반드시 endInsertion() 함수를 호출해야 합니다.

[**Syntax**]
void beginInsertion(int timeout) throws JposException

[**Parameters**]
• int timeout : 카드 삽입 체크 시간 설정

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(5000);
	SmartCardRW.setDeviceEnabled(true);

	smartCardRW. beginInsertion(5000);
	smartCardRW.endInsertion();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-9 endInsertion()

삽입된 Smart Card chip에 파워를 인가합니다.
해당 함수 호출 전 반드시 beginInsertion 함수를 호출해야 합니다.

[**Syntax**]
void endInsertion() throws JposException

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(5000);
	SmartCardRW.setDeviceEnabled(true);

	smartCardRW. beginInsertion(5000);
	smartCardRW.endInsertion();}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-10 beginRemoval()

Smart Card chip에 인가된 파워를 해지합니다.
해당 함수 호출 후 반드시 endRemoval 함수를 호출해야 합니다.

[**Syntax**]
void beginRemoval(int timeout) throws JposException

[**Parameters**]
• int timeout : 파워 해지 시간 설정

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(5000);
	SmartCardRW.setDeviceEnabled(true);

	smartCardRW.beginRemoval(5000);
	smartCardRW.endRemoval();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-11 endRemoval()

Smart Card chip에 파워가 정상적으로 해지 되었는지 체크 합니다.
해당 함수 호출 전 반드시 beginRemoval 함수를 호출해야 합니다.

[**Syntax**]
void endRemoval() throws JposException

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open("SmartCardRW");
	smartCardRW.claim(5000);
	SmartCardRW.setDeviceEnabled(true);

	smartCardRW.beginRemoval(5000);
	smartCardRW.endRemoval();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-4-12 readData()

Smart Card chip에 데이터를 R/W 합니다.

[**Syntax**]
void readData(int action, int[] count, String[] data) throws JposException

[**Parameters**]
• int action : 고정값 SmartCardRWConst.SC_READ_DATA
• int[] count : 응답 데이터 사이즈
• String[] data : R/W 버퍼(“cp437” 인코딩)

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.SmartCardRW;
import jpos.SmartCardRWConst;

try
{
	SmartCardRW smartCardRW = new SmartCardRW();
	smartCardRW.open(“SmartCardRW”);
	smartCardRW.claim(3000);
	smartCardRW.setDeviceEnabled(true);

	// Power up
	smartCardRW.beingInsertion(5000);
	smartCardRW.endInsertion();

	String[] data = new String[]{
		new String(new byte[]{
			0x00, (byte) 0xA4, 0x04, 0x00, 0x07, (byte) 0xD4, 0x10, 0x65,
			0x09, (byte) 0x90, 0x00, 0x10}, "CP437")
	int[] count = new int[1];

	smartCardRW. readData(SmartCardRWConst.SC_READ_DATA, count, data);
	int rspSize = count[0];
	byte[] rspData = data[0].getBytes("CP437");

	// Power down
	smartCardRW.beginRemoval(5000);
	smartCardRW.endRemoval();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5 CashDrawer Class

• CashDrawer 제어를 위한 Class 입니다. 특정 기능 수행 중 에러 발생 시 JposException을 발생시킵니다. (“5-1 JposException” 참조)

6-5-1 open()

CashDrawer 클래스 사용을 시작하며 메모리 할당 등의 초기화 작업이 포함 되어있습니다. Claim 이상의 Method 호출을 위해서 반드시 선행 되어야 합니다. BXLConfigLoader Class를 통해 저장되지 않은 장치는 open되지 않습니다.

[**Syntax**]
void open(String logicalDeviceName) throws JposException

[**Parameters**]
• String logicalDeviceName : Open 하려는 장치 이름(CashDrawer)

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	CashDrawer cashDrawer = new CashDrawer();
	cashDrawer.open("CashDrawer");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5-2 claim()

Device 정보에 포함되어있는 Port 를 실제로 Open 하기 위해 시도 하며, 메모리 할당 초기화 등의 초기화 작업이 일부 포함되어있습니다.
Device 사용을 활성화 하기 위해서 반드시 선행 되어야 합니다.

[**Syntax**]
void claim(int timeout) throws JposException

[**Parameters**]
• int timeout : 이 파라미터에서 지정한 시간동안 Port open을 시도합니다.

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	CashDrawer cashDrawer = new CashDrawer();
	cashDrawer.open("CashDrawer");
	cashDrawer.claim(3000);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5-3 setDeviceEnabled()

장치 사용 여부를 포함하고 있습니다.
DeviceEnabled 의 값이 비활성화 되어 있으면 기능 이용이 불가능 할 수 있습니다.

[**Syntax**]
void setDeviceEnabled(boolean deviceEnabled) throws JposException

[**Parameters**]
• boolean deviceEnabled : 장치 활성화 상태를 입력 합니다..

값	설명
true	활성화
false	비활성화

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	CashDrawer cashDrawer = new CashDrawer();
	cashDrawer.open("CashDrawer");
	cashDrawer.claim(3000);
	cashDrawer.setDeviceEnabled(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5-4 release()

Claim 된 Device의 Port 사용을 물리적으로 종료 합니다.
메모리 해제 등의 작업이 수행 될 수 있습니다

[**Syntax**]
void release() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	cashDrawer.release();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5-5 close()

Open 된 Device의 사용을 종료 합니다.
메모리 해제 등의 작업이 일부 수행 될 수 있습니다.

[**Syntax**]
void close() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	cashDrawer.close();
	cashDrawer.setDeviceEnabled(false);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5-6 openDrawer()

Cash Drawer를 Open 합니다.

[**Syntax**]
void openDrawer() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	CashDrawer cashDrawer = new CashDrawer();
	cashDrawer.open("CashDrawer");
	cashDrawer.claim(3000);
	cashDrawer.setDeviceEnabled(true);

	cashDrawer.openDrawer();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-5-7 getDrawerOpened()

Cash Drawer Open 상태를 리턴 합니다.

[**Syntax**]
boolean getDrawerOpened() throws JposException

[**Return Values**]

값	설명
true	Cash drawer is opened.
false	Cash drawer is closed.

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.CashDrawer;
import jpos.CashDrawerConst;

try
{
	CashDrawer cashDrawer = new CashDrawer();
	cashDrawer.open("CashDrawer");
	cashDrawer.claim(3000);
	cashDrawer.setDeviceEnabled(true);

	if(cashDrawer.getDrawerOpened())
	{
		// opened
	}
	else
	{
		// closed
	}
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6 LocalSmartCardRW Class

• SPP-R215 프린터의 SCR(Smart Card Reader) 제어를 위한 Class 입니다. 특정 기능 수행 중 에러 발생 시 JposException을 발생시킵니다. (“5-1 JposException” 참조)

주의	LocalSmartCardRW 클래스는 SPP-R215 오프라인 모델에서만 사용할 수 있습니다.

6-6-1 open()

LocalSmartCardRW 클래스 사용을 시작하며 메모리 할당 등의 초기화 작업이 포함되어있습니다.
Claim 이상의 Method 호출을 위해서 반드시 선행 되어야 합니다.
BXLConfigLoader Class를 통해 저장되지 않은 장치는 open되지 않습니다.

[**Syntax**]
void open(String logicalDeviceName) throws JposException

[**Parameters**]
• String logicalDeviceName : Open 하려는 장치 이름(LocalSmartCardRW)

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-2 claim()

Device 정보에 포함되어있는 Port 를 실제로 Open 하기 위해 시도 하며, 메모리 할당 초기화 등의 초기화 작업이 일부 포함되어있습니다.
Device 사용을 활성화 하기 위해서 반드시 선행 되어야 합니다.

[**Syntax**]
void claim(int timeout) throws JposException

[**Parameters**]
• int timeout : 이 파라미터에서 지정한 시간동안 Port open을 시도합니다.

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
	localSmartCardRW.claim(3000);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-3 setDeviceEnabled()

장치 사용 여부를 포함하고 있습니다.
DeviceEnabled 의 값이 비활성화 되어 있으면 기능 이용이 불가능 할 수 있습니다.

[**Syntax**]
void setDeviceEnabled(boolean deviceEnabled) throws JposException

[**Parameters**]
• boolean deviceEnabled : 장치 활성화 상태를 입력 합니다..

값	설명
true	활성화
false	비활성화

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
	localSmartCardRW.claim(3000);
	localSmartCardRW.setDeviceEnabled(true);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-4 release()

Claim 된 Device의 Port 사용을 물리적으로 종료 합니다.
메모리 해제 등의 작업이 수행 될 수 있습니다

[**Syntax**]
void release() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	localSmartCardRW.release();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-5 close()

Open 된 Device의 사용을 종료 합니다.
메모리 해제 등의 작업이 일부 수행 될 수 있습니다.

[**Syntax**]
void close() throws JposException

[Example]
```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	localSmartCardRW.close();
	localSmartCardRW.setDeviceEnabled(false);
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-6 getCardInfo()

SPP-R215 SCR 모듈에 카드 정보를(카드번호, 유효기간) 요청합니다.

[**Syntax**]
void getCardInfo(int timeout) throws JposException

[**Parameters**]
• int timeout : 카드 리딩 타입아웃을 설정합니다.(초단위)

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
	localSmartCardRW.claim(3000);
	localSmartCardRW.setDeviceEnabled(true);

	localSmartCardRW.getCardInfo(10); // 타임아웃 10초
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-7 getTrack2()

SPP-R215 SCR 모듈로부터 읽어온 Track2 데이터를 리턴합니다.

주의	카드 번호를 얻어오기 위해서는 getCardInfo를 선행해야 합니다.

[**Syntax**]
byte[] getTrack2() throws JposException

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
	localSmartCardRW.claim(3000);
	localSmartCardRW.setDeviceEnabled(true);

	localSmartCardRW.getCardInfo(10); // 타임아웃 10초
	byte[] track2 = localSmartCardRW.getTrack2();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-8 getCardNumber()

SPP-R215 SCR 모듈로부터 읽어온 카드 번호를 리턴합니다.

주의	카드 번호를 얻어오기 위해서는 getCardInfo를 선행해야 합니다.

[**Syntax**]
byte[] getCardNumber() throws JposException

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
	localSmartCardRW.claim(3000);
	localSmartCardRW.setDeviceEnabled(true);

	localSmartCardRW.getCardInfo(10); // 타임아웃 10초
	byte[] cardNumber = localSmartCardRW.getCardNumber();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

6-6-9 getCardDueDate()

SPP-R215 SCR 모듈로부터 읽어온 카드 유효기간을 리턴합니다.

주의	카드 유효기간을 얻어오기 위해서는 getCardInfo를 선행해야 합니다.

[**Syntax**]
byte[] getCardDueDate() throws JposException

[Example]

```
import jpos.JposConst;
import jpos.JposException;
import jpos.LocalSmartCardRW;
import jpos.LocalSmartCardRWConst;

try
{
	LocalSmartCardRW localSmartCardRW = new LocalSmartCardRW();
	localSmartCardRW.open("LocalSmartCardRW");
	localSmartCardRW.claim(3000);
	localSmartCardRW.setDeviceEnabled(true);

	localSmartCardRW.getCardInfo(10); // 타임아웃 10초
	byte[] cardNumber = localSmartCardRW.getCardNumber();
	byte[] cardDueDate = localSmartCardRW.getCardDueDate();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

7. 사용 예제

7-1 텍스트 인쇄

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	// BXLConfigLoader 생성 / 설정 파일 open
	BXLConfigLoader bxlConfigLoader = new BXLConfigLoader(this);
	bxlConfigLoader.openFile();
}
catch(JposException e)
{
	e.printStackTrace();
	bxlConfigLoader.newFile();
}

try
{
	// 장치 정보 추가
	bxlConfigLoader.addEntry("SPP-R200III",
					BXLConfigLoader.DEVICE_CATEGORY_POS_PRINTER,
					BXLConfigLoader.PRODUCT_NAME_SPP_R200III,
					BXLConfigLoader.DEVICE_BUS_BLUETOOTH,
					"74:F0:7D:E4:11:AF");
	// 설정 파일 저장
	bxlConfigLoader.saveFile();

	// 프린터 Open/Claim/DeviceEnabled
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	// 텍스트 인쇄
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT, "Print Data\n");
	posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT,
				EscapeSequence.getString(7) + "www.bixolon.com\n");

	posPrinter.close();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

7-2 이미지 인쇄

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
	// BXLConfigLoader 생성 / 설정 파일 open
	BXLConfigLoader bxlConfigLoader = new BXLConfigLoader(this);
	bxlConfigLoader.openFile();
}
catch(JposException e)
{
	e.printStackTrace();
	bxlConfigLoader.newFile();
}

try
{
	// 장치 정보 추가
	bxlConfigLoader.addEntry("SPP-R200III",
					BXLConfigLoader.DEVICE_CATEGORY_POS_PRINTER,
					BXLConfigLoader.PRODUCT_NAME_SPP_R200III,
					BXLConfigLoader.DEVICE_BUS_BLUETOOTH,
					"74:F0:7D:E4:11:AF");
	// 설정 파일 저장
	bxlConfigLoader.saveFile();

	// 프린터 Open/Claim/DeviceEnabled
	POSPrinter posPrinter = new POSPrinter(this);
	posPrinter.open("SPP-R200III");
	posPrinter.claim(5000);
	posPrinter.setDeviceEnabled(true);

	// 이미지 인쇄
	ByteBuffer buffer = ByteBuffer.allocate(4);
	buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
	buffer.put((byte) 80); // brightness
	buffer.put((byte) 0x01); // compress
	buffer.put((byte) 0x00);

	posPrinter.printBitmap(buffer.getInt(0),
				"/storage/emulated/0/kicc/sample_image.png",
				384,
				POSPrinterConst.PTR_BM_LEFT);

	posPrinter.close();
}
catch(JposException e)
{
	e.printStackTrace();
}
```

7-3 Page mode 인쇄

```
import com.bxl.config.editor.BXLConfigLoader;
import jpos.config.JposEntry;
import jpos.POSPrinter;
import jpos.POSPrinterConst;

try
{
    // BXLConfigLoader 생성 / 설정 파일 open
    BXLConfigLoader bxlConfigLoader = new BXLConfigLoader(this);
    bxlConfigLoader.openFile();
}
catch(JposException e)
{
    e.printStackTrace();
    bxlConfigLoader.newFile();
}

try
{
    // 장치 정보 추가
    bxlConfigLoader.addEntry("SPP-R200III",
                             BXLConfigLoader.DEVICE_CATEGORY_POS_PRINTER,
                             BXLConfigLoader.PRODUCT_NAME_SPP_R200III,
                             BXLConfigLoader.DEVICE_BUS_BLUETOOTH,
                             "74:F0:7D:E4:11:AF");
    // 설정 파일 저장
    bxlConfigLoader.saveFile();

    // 프린터 Open/Claim/DeviceEnabled
    POSPrinter posPrinter = new POSPrinter(this);
    posPrinter.open("SPP-R200III");
    posPrinter.claim(5000);
    posPrinter.setDeviceEnabled(true);

    ByteBuffer buffer = ByteBuffer.allocate(4);
    buffer.put((byte) POSPrinterConst.PTR_S_RECEIPT);
    buffer.put((byte) 80); // brightness
    buffer.put((byte) 0x01); // compress
    buffer.put((byte) 0x00);

    // Page mode 인쇄 영역 설정
    posPrinter.setPageModePrintArea("0,0,384,1200");
    // Page mode 인쇄 방향 설정
    posPrinter.setPageModePrintDirection(POSPrinterConst.PTR_PD_LEFT_TO_RIGHT);
    // Page mode 전환
    posPrinter.pageModePrint(POSPrinterConst.PTR_PM_PAGE_MODE);

    // 인쇄 위치 설정 : 가로/세로
    posPrinter.setPageModeHorizontalPosition(0);
    posPrinter.setPageModeVerticalPosition(0);
    posPrinter.printBitmap(buffer.getInt(0),
                           "/storage/emulated/0/kicc/sample_image.png",
                           384,
                           POSPrinterConst.PTR_BM_LEFT);
```

```
    posPrinter.setPageModeHorizontalPosition(130);
    posPrinter.setPageModeVerticalPosition(200);
    posPrinter.printNormal(POSPrinterConst.PTR_S_RECEIPT,
                           EscapeSequence.getString(0) + // Normal
                           EscapeSequence.getString(7) + // Bold
                           EscapeSequence.getString(18) + //Scale_2_time_horizontally
                           EscapeSequence.getString(26) + // Scale_2_time_vertically
                           "www.bixolon.com\n");

    posPrinter.setPageModeHorizontalPosition(340);
    posPrinter.setPageModeVerticalPosition(550);
    String data = "www.bixolon.com\n";
    posPrinter.printBarCode(POSPrinterConst.PTR_S_RECEIPT,
                            data,
                            POSPrinterConst.PTR_BCS_QRCODE,
                            0, 6/*1to8*/,
                            POSPrinterConst.PTR_BC_LEFT,
                            POSPrinterConst.PTR_BC_TEXT_NONE);

    // 페이지 모드 인쇄 시작
    posPrinter.pageModePrint(POSPrinterConst.PTR_PM_NORMAL);

    posPrinter.close();
}
catch(JposException e)
{
    e.printStackTrace();
}
```

저작권

© BIXOLON Co., Ltd. 모든 권한을 소유합니다.

이 사용설명서와 제품에 사용된 저작물은 저작권법에 의해 보호되어 있습니다.
(주)빅솔론의 사전 서면 동의 없이 사용 설명서 및 제품에 사용된 저작물에 대한 일부 또는 전체를 무단으로 복제, 저장, 전송하는 것을 금합니다.
제공된 정보는 본 제품에만 해당되며 다른 제품에 대해서는 적용되지 않습니다.
또한 본 정보 사용으로 인해 발생하는 직/간접적 손해에 대해 책임지지 않습니다.

- 빅솔론 로고는 (주)빅솔론의 등록상표 입니다.
- 모든 다른 상표 또는 제품 이름은 해당하는 회사 또는 조직의 상표입니다.

(주)빅솔론은 제품의 기능과 품질 향상을 위하여 지속적인 개선을 하고 있습니다.
이로 인하여 제품의 사양과 매뉴얼의 내용은 사전 통보 없이 변경될 수 있습니다.

사용시 주의사항

프린터와 같은 전자 제품은 정전기에 의해 쉽게 훼손될 수 있습니다.
정전기로부터 프린터를 보호하기 위해서는 프린터 후면 부에 케이블을 연결하거나 제거하기 전에 반드시 프린터 전원을 끄십시오. 만약 프린터가 정전기로부터 손상을 입었을 경우에는 가까운 구입처에 문의하십시오.

개정 이력

Rev.	Date	Page	Description
2.00	17.08.21	-	신규제작
2.01	17.11.22	4,11,13,23 24, 40,41,43,44	SRP-QE300/SRP-QE302 모델추가, 누락내용추가
2.02	18.01.09	4,11,14,16,18,23, 26,30,51,52,53,54,55	SRP-E300/SRP-E302 모델추가. BCD-3000 제어 API 설명 추가. 배터리 상태 이벤트 추가 외
2.03	18.05.28	15,17,19,58,59, 76~80	샘플 애플리케이션 변경 및 레퍼런스 클래스 추가. SmartCard 제어 API 설명 추가. 누락내용추가.
2.04	18.08.30	1,5,10,13,19,20,23,68, 69,70,72,73,79,81,83	SRP-383 모델추가. 누락내용추가
2.05	18.09.12	1, 5, 18, 19, 22	BK3-3 모델추가. FeedCut 설명 추가.
2.06	19.03.08	1 ~ 5, 22, 57, 58 ~ 83, 84 ~ 90	SRP-Q200 모델추가. NV 이미지 다운로드 API 추가. SPP-R215 모델 지원. SCR API 추가(LocalSmartCardRW). 예제코드 오타 수정.
2.07	19.09.26	1 ~ 5, 58, 59	printSvg API 설명 추가 updateFirmware API 설명 추가 printPdf 미지원